A. Additional results on self-speculative decoding

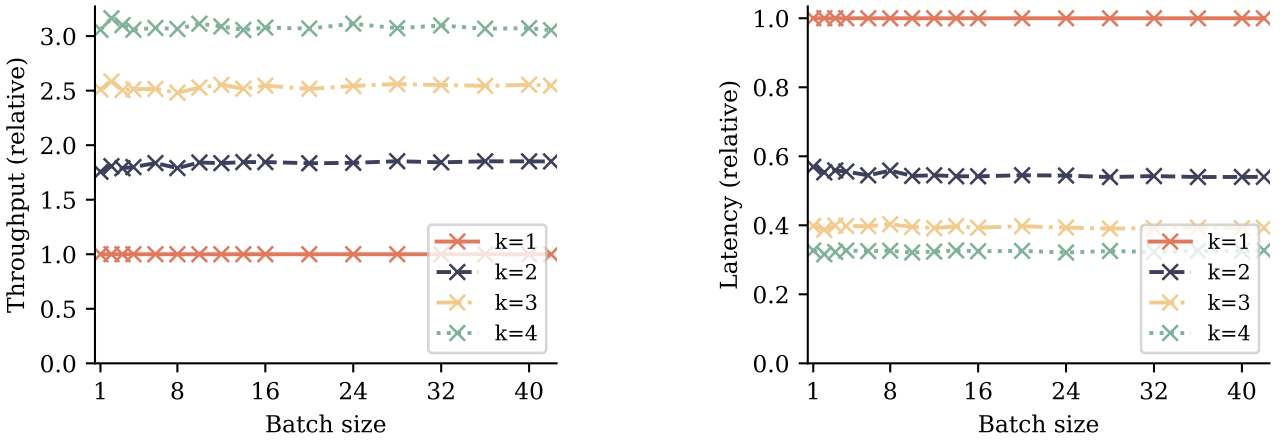


Figure S10: **Decoding speeds and latencies with self-speculative decoding relative to standard autoregressive decoding.** We use k heads of a 4-token prediction model and evaluate decoding speeds of a code model as explained in Table S2. All numbers are relative to the autoregressive ($k = 1$) baseline with the same batch size.

Table S2: **Relative speedups with self-speculative decoding.** For wikipedia and books we prompt a 7B parameter model trained on 500B tokens, and for code we prompt a 7B parameter model trained on 1T tokens of code on 4200 sequences of 512 tokens from a test dataset not seen during training, and generate completions consisting of 512 tokens using greedy self-speculative decoding (Stern et al., 2018) using the indicated number of heads from a 4-token prediction model. Note that the maximal speedup that can be obtained with self-speculative decoding using k heads is k. The last column shows the average number of tokens retrieved from a forward containing this sequence (both verification and prediction). The speedup was evaluated at the maximal batch size of 42, but is constant across batch sizes (Figure S10).

	Wikipedia		Books		Code	
# Heads used	Rel. speedup	Tokens / forward	Rel. speedup	Tokens / forward	Rel. speedup	Tokens / forward
1	1.00	1.00	1.00	1.00	1.00	1.00
2	1.79	1.88	1.77	1.87	1.85	1.94
3	2.35	2.57	2.32	2.56	2.54	2.78
4	2.74	3.12	2.67	3.09	**3.05**	**3.50**

Table S3: **Relative speedups with self-speculative decoding with byte-level models on code.** We prompt the 7B parameter models from Section 3.3 on 4096 sequences of 1024 bytes of code not seen during training, and generate completions consisting of 1024 bytes using greedy self-speculative decoding (Stern et al., 2018) as in Table S2. The speedup was evaluated at a batch size of 16.

	$n = 8$		$n = 16$		$n = 32$	
# Heads used	Rel. speedup	Tokens / forward	Rel. speedup	Tokens / forward	Rel. speedup	tokens / forward
1	1.00	1.00	1.00	1.00	1.00	1.00
2	1.94	1.98	1.94	1.98	1.93	1.97
4	3.67	3.84	3.63	3.81	3.62	3.80
8	6.39	7.04	6.25	6.92	6.22	6.89
12	–	–	8.07	9.36	8.01	9.30
16	–	–	9.24	11.20	9.15	11.15
20	–	–	–	–	9.83	12.61
24	–	–	–	–	10.34	13.67
28	–	–	–	–	10.55	14.58
32	–	–	–	–	10.84	15.35

B. Alternative architectures

Table S4: **Alternative architectures improve on baseline but not as consistently.** Alternative architectures for multi-token prediction are worth exploring to improve efficiency. Here we tried Anticausal, causal and linear and showed no significant improvement with respect to Parallel architecture.

				MBPP			HumanEval			APPS/Intro		
n	Head type	Architecture	+Layers	@1	@10	@100	@1	@10	@100	@1	@10	@100
1	transformer	parallel	0	30.0	53.8	73.7	22.8	36.4	62.0	2.8	7.8	17.4
4	linear	parallel	0	33.6	55.0	76.2	21.9	38.5	63.7	3.1	10.1	23.0
	transformer	anticausal	0	30.8	54.8	75.3	20.9	38.4	64.5	2.0	8.7	21.6
		causal	0	31.9	54.9	74.9	20.9	38.1	67.3	4.0	11.6	22.8
		parallel	0	33.8	55.9	76.9	24.0	40.1	66.1	1.6	7.1	19.9
			3	33.3	55.7	77.3	22.4	39.4	66.7	2.6	9.5	22.1

The architecture described in Section 2 is not the only sensible option, but proved technically viable and well-performing in our experiments. We describe and compare alternative architectures in this section.

Replicated unembeddings Replicating the unembedding matrix n times is a simple method for implementing multi-token prediction architectures. However, it requires matrices with shapes (d, nV) in the notation of Section 2, which is prohibitive for large-scale trainings.

Linear heads Apart from using a single transformer layer for the heads H_i, other architectures are conceivable. We experimented with a single linear layer without any nonlinearity as heads, amounting to linear probing of the model's residual representation z. Architectures with more than one layer per head are also possible, but we did not pursue this direction further.

Causal and anticausal variant Instead of making the prediction heads $P_i(x_{t+i} \mid z_{t:1})$ architecturally independent of each other, we can also allow them to rely on other heads' (pre-unembedding) outputs. In a *causal* variant, later prediction heads are applied on top of the previous ones, i.e. the i-th prediction head P_i is given by

$$P_\theta(x_{t+i}|\cdot) = \text{softmax} \circ f_u \circ f_{h_i} \circ f_{h_{i-1}} \cdots \circ f_{h_1} \circ f_s.$$

In another *anticausal* variant, the network starts by predicting the most distant tokens before gradually refining up to the following token:

$$P_\theta(x_{t+i}|\cdot) = \text{softmax} \circ f_u \circ f_{h_i} \circ f_{h_{i+1}} \cdots \circ f_{h_n} \circ f_s.$$

These architectures likewise allow a sequential forward/backward order as the parallel architecture from Section 2. This is described in Figure S11.

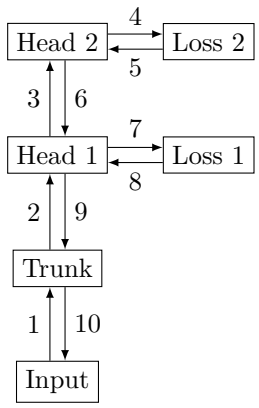


Figure S11: **Order of the forward/backward in a causal n-token prediction model with $n = 2$ heads.** Like in the forward/backward depicted for parallel prediction heads in Figure 2, we avoid materializing all unembedding layer gradients in memory simultaneously and reduce peak GPU memory usage significantly. The iteration over the heads starts with the one furthest to the trunk. At each head, a gradient from the succeeding prediction heads and from the head's own loss are accumulated for both the head's output and its weights.

C. Training speeds

Table S5: **Training time relative to next-token prediction training.** The slight overhead when using multi-token prediction here is explained by a suboptimal use of Fully Sharded Data Parallel. In our implementation, when doing separate backward passes for each head, we lose the overlap of layer weight communication and computation, therefore it incurs a very slight overhead that can be removed if reimplemented correctly.

Model	n=1	n=2	n=4
0.3B	1.00	1.07	1.22
0.6B	1.00	1.05	1.13
1.3B	1.00	1.04	1.12
3B	1.00	1.02	1.07
6.7B	1.00	1.02	1.07
13B	1.00	1.04	1.09

D. Finetuning

Table S6: **Finetuning LLama 2 with multi-token prediction does not significantly improve performance.** We tried to finetune LLama 2 with 4-token prediction but this did not yield significant improvements compared to the baseline. We suppose that this new loss changes the initialization too brutally and never really recovers. We still some improvements for example on MBPP Pass@1. All runs use 200B tokens of code.

			MBPP			HumanEval			APPS/Intro		
n	Head type	+Layers	@1	@10	@100	@1	@10	@100	@1	@10	@100
1	transformer	0	39.6	65.1	82.4	31.4	57.7	84.7	10.0	21.6	36.7
4	linear	0	39.3	63.7	81.3	29.0	53.4	82.2	6.9	20.0	34.0
	transformer	0	38.3	62.2	80.1	27.9	53.6	82.4	5.8	18.2	34.3
		3	42.5	64.4	81.3	28.7	56.9	82.4	7.8	21.2	37.3

E. Additional results on model scaling behavior

Table S7: **Scaling model size** Full results of scaling model size with n=1,2 and 4.

		MBPP			HumanEval		
Model Size	Fut	@1	@10	@100	@1	@10	@100
0.3B	1	1.8	10.4	29.9	1.9	5.0	10.9
	2	1.7	10.1	27.2	1.5	4.4	10.3
	4	1.0	6.3	20.1	1.2	4.0	8.6
0.6B	1	4.7	21.0	45.2	2.9	8.5	16.7
	2	4.6	21.0	44.7	3.2	8.9	16.2
	4	3.0	15.6	38.0	2.7	7.7	15.5
1.3B	1	6.8	27.0	51.0	4.6	13.1	24.3
	2	7.3	27.5	51.7	5.4	13.6	23.3
	4	7.4	27.6	50.1	4.8	12.3	22.5
3B	1	11.1	36.4	60.4	7.2	17.2	29.8
	2	11.8	37.2	60.5	8.0	18.2	31.2
	4	12.7	37.6	61.1	7.2	18.5	33.3
6.7B	1	23.9	54.2	74.7	12.8	29.3	51.7
	2	24.7	54.8	76.4	13.2	32.2	53.9
	4	26.0	55.8	76.0	13.8	33.2	58.5
13B	1	26.0	57.1	77.0	14.1	33.6	56.0
	2	30.5	60.5	79.4	15.2	36.9	60.0
	4	30.5	61.0	79.2	15.8	38.6	63.5

F. Details on CodeContests finetuning

We use the Python subset of the CodeContests (Li et al., 2022) train split with reward annotations ("correct" / "incorrect") and condition on correct solutions at evaluation time. For evaluation, we generate 1000 samples per problem from the test split for each temperature $T \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$, and compute the unbiased estimator for pass@k from Chen et al. (2021) for each value of k and T. It is possible that models that were pretrained with different losses have different respective optimal temperatures for pass@k, so we compute and show $k \mapsto \max_T \text{pass_at}(k, T)$ in Figure 4. In other words, we grant pass@k access to a temperature oracle. For small values of k, pass@k measures the capability of understanding and solving tasks while for large k, it additionally favors diversity in outputs. According to the results in Figure 4, multi-token prediction pretraining leads to finetuned models that are better on both axes.